



MyMediCloud MMC - Technische Dokumentation

!MyMediCloud MMC Logo

Version 1.0

Stand: Januar 2026

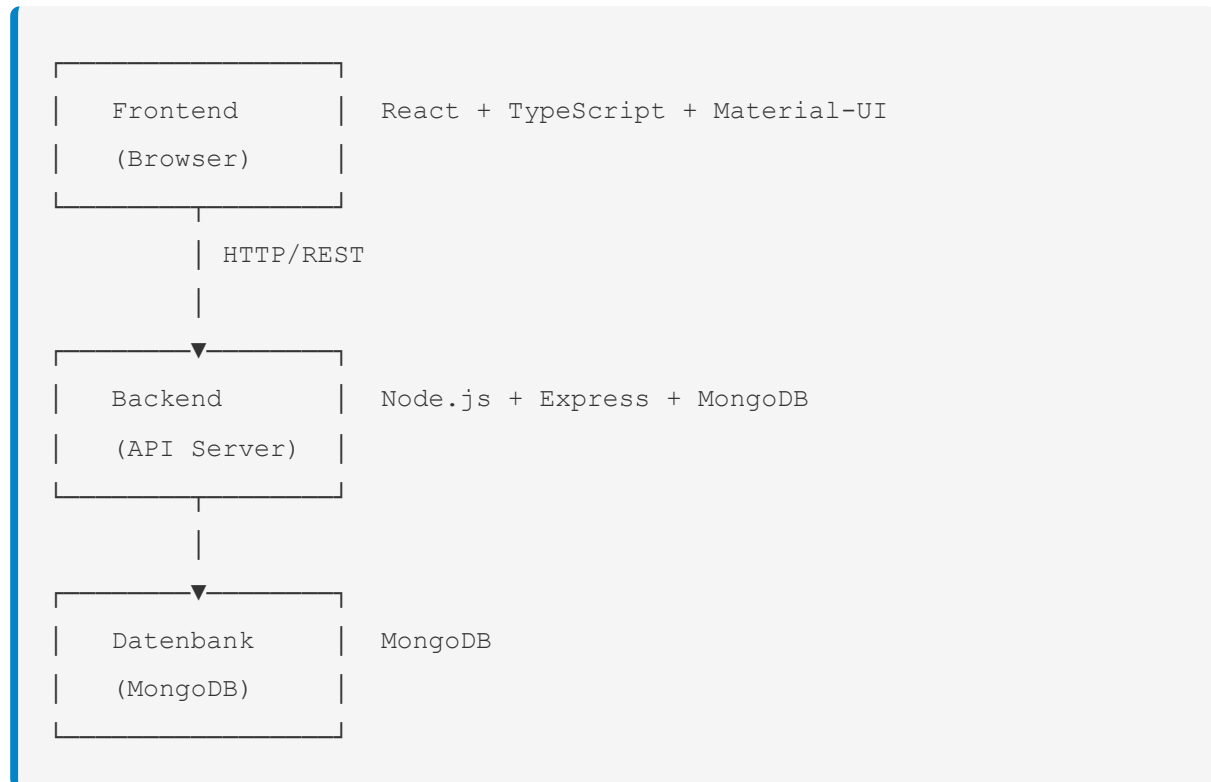
Inhaltsverzeichnis

1. Systemarchitektur
 2. Installation und Setup
 3. Backend-Architektur
 4. Frontend-Architektur
 5. Datenbank-Schema
 6. API-Dokumentation
 7. Sicherheit
 8. Integrationen
 9. Deployment
 10. Wartung und Updates
-

1. Systemarchitektur

1.1 Übersicht

MyMediCloud MMC ist eine **moderne, webbasierte Anwendung** mit einer **3-Tier-Architektur**:



1.2 Technologie-Stack

Frontend:

- React 19.2.0
- TypeScript 4.9.5
- Material-UI (MUI) 7.3.4
- Redux Toolkit 2.9.1
- React Router 7.9.4

- Axios 1.12.2

Backend:

- Node.js (v16+)
- Express.js
- MongoDB mit Mongoose
- JWT für Authentifizierung
- bcryptjs für Passwort-Hashing

Entwicklungstools:

- ESLint für Code-Qualität
- Prettier für Code-Formatierung
- Git für Versionskontrolle

1.3 Systemanforderungen

Server:

- Node.js v16 oder höher
- MongoDB 4.4 oder höher
- Mindestens 2 GB RAM

- 10 GB freier Speicherplatz

Client:

- Moderne Browser (Chrome, Firefox, Safari, Edge)
- JavaScript aktiviert
- Internetverbindung

2. Installation und Setup

2.1 Voraussetzungen

1. Node.js installieren

```
bash
# Prüfen Sie die Version
node --version # Sollte v16 oder höher sein
npm --version
```

2. MongoDB installieren

- Lokale Installation: <https://www.mongodb.com/try/download/community>
- Oder MongoDB Atlas (Cloud): <https://www.mongodb.com/cloud/atlas>

3. Git installieren

```
bash
git --version
```

2.2 Projekt klonen

```
bash
git clone [repository-url]
cd ordinationssoftware-at
```

2.3 Backend Setup

```
bash
cd backend
npm install
```

Umgebungsvariablen konfigurieren

```
cp .env.example .env
```

Bearbeiten Sie .env mit Ihren Eins

.env Datei konfigurieren:

```
env
PORT=5001
MONGODB_URI=mongodb://localhost:27017/ordinationssoftware
JWT_SECRET=your-secret-key-here
NODE_ENV=development
```

Backend starten:

```
bash  
npm start
```

Oder für Entwicklung mit Auto-Inst

```
npm run dev
```

2.4 Frontend Setup

```
bash  
cd frontend  
npm install
```

Umgebungsvariablen (optional)

```
cp .env.example .env
```

Bearbeiten Sie .env falls nötig

.env Datei (optional):

```
env  
REACT_APP_API_URL=http://localhost:5001
```

Frontend starten:

```
bash  
npm start
```

Öffnet automatisch [http://localhost](http://localhost:3000)

2.5 Datenbank initialisieren

```
bash  
cd backend
```

ICD-10 Daten importieren

```
node scripts/importAustrianIcd10.js
```

Demo-Benutzer erstellen

```
node scripts/setupDemoUser.js
```

Demo-Daten erstellen

```
node scripts/setupDemoStaffData.js  
node scripts/setupDemoResources.js
```

2.6 Erster Administrator

Super-Admin Setup:

1. Öffnen Sie <http://localhost:3000/super-admin-setup>
2. Erstellen Sie den ersten Administrator
3. Melden Sie sich mit diesem Account an

3. Backend-Architektur

3.1 Projektstruktur


```

backend/
├── controllers/           # API Controller
│   ├── authController.js
│   ├── patientController.js
│   ├── appointmentController.js
│   └── ...
├── models/               # MongoDB Modelle
│   ├── User.js
│   ├── Patient.js
│   ├── Appointment.js
│   └── ...
├── routes/               # Express Routes
│   ├── auth.js
│   ├── patients.js
│   ├── appointments.js
│   └── ...
├── middleware/           # Middleware
│   ├── auth.js
│   ├── errorHandler.js
│   └── ...
├── services/             # Business Logic
│   ├── billingService.js
│   ├── connectors/
│   │   ├── insuranceConnector.js
│   │   ├── paymentConnector.js
│   │   └── ...
│   └── ...
├── utils/                # Hilfsfunktionen
│   ├── billing-calculator.js
│   ├── validation.js
│   └── ...
├── scripts/              # Setup & Import Scripts
│   ├── importAustrianIcd10.js
│   ├── setupDemoUser.js
│   └── ...
└── server.js             # Hauptserver-Datei

```

3.2 API-Struktur

RESTful API:

- Alle API-Endpunkte beginnen mit `/api/`
- Authentifizierung über JWT-Token
- JSON als Datenformat
- Standard HTTP-Status-Codes

Beispiel-Endpunkte:

```
POST    /api/auth/login
POST    /api/auth/register
GET     /api/patients
POST    /api/patients
PUT     /api/patients/:id
DELETE  /api/patients/:id
```

3.3 Middleware

Authentifizierung:

- JWT-Token wird im Header übermittelt: `Authorization: Bearer`
- Token wird bei jeder Anfrage validiert
- Ungültige Token führen zu 401 Unauthorized

Fehlerbehandlung:

- Zentrale Fehlerbehandlung
- Strukturierte Fehlerantworten

- Logging aller Fehler

3.4 Datenbank-Modelle

Hauptmodelle:

- `User` : Benutzer und Personal
- `PatientExtended` : Patienten mit erweiterten Daten
- `Appointment` : Termine
- `Document` : Medizinische Dokumente
- `Invoice` : Rechnungen
- `ServiceCatalog` : Leistungskatalog
- `Location` : Standorte
- `Room` : Räume
- `Device` : Geräte

4. Frontend-Architektur

4.1 Projektstruktur

```

frontend/
├── src/
│   ├── components/      # React Komponenten
│   │   ├── Layout/
│   │   ├── PatientSidebar.tsx
│   │   ├── DiagnosisManager.tsx
│   │   └── ...
│   ├── pages/           # Hauptseiten
│   │   ├── Dashboard.tsx
│   │   ├── Patients.tsx
│   │   ├── Appointments.tsx
│   │   └── ...
│   ├── store/           # Redux Store
│   │   ├── slices/
│   │   │   ├── patientSlice.ts
│   │   │   ├── appointmentSlice.ts
│   │   │   └── ...
│   │   └── hooks.ts
│   ├── utils/           # Hilfsfunktionen
│   │   ├── api.ts
│   │   ├── validation.ts
│   │   └── ...
│   ├── types/           # TypeScript Typen
│   │   └── ...
│   └── App.tsx           # Hauptkomponente
├── public/              # Statische Assets
└── package.json

```

4.2 State Management

Redux Toolkit:

- Zentrale State-Verwaltung
- Slices für verschiedene Domänen
- Async Actions mit createAsyncThunk

Beispiel Slice:

```
typescript
const patientSlice = createSlice({
  name: 'patients',
  initialState: { patients: [], loading: false },
  reducers: { ... },
  extraReducers: (builder) => { ... }
});
```

4.3 Routing

React Router:

- Client-Side Routing
- Protected Routes für authentifizierte Bereiche
- Dynamische Routen für Details

Beispiel Route:

```
typescript
} />
```

4.4 API-Integration

Axios:

- Zentrale API-Instanz in `utils/api.ts`
- Automatische Token-Übertragung
- Fehlerbehandlung

Beispiel API-Call:

```
typescript
const response = await api.get('/patients');
```

5. Datenbank-Schema

5.1 Hauptmodelle

User:

```
javascript
{
  email: String (unique, required),
  password: String (hashed, required),
  firstName: String,
  lastName: String,
  role: String (enum),
  isActive: Boolean,
  twoFactorEnabled: Boolean,
  ...
}
```

PatientExtended:

```
javascript
{
  firstName: String (required),
  lastName: String (required),
  dateOfBirth: Date (required),
  gender: String (required),
  address: {
    street: String,
    city: String,
    zipCode: String,
    country: String
  },
  insuranceProvider: String,
  insuranceNumber: String,
  medicalHistory: Array,
  allergies: Array,
  currentMedications: Array,
  ...
}
```

Appointment:

```
javascript
{
  patient: ObjectId (ref: PatientExtended),
  doctor: ObjectId (ref: User),
  startTime: Date (required),
  endTime: Date (required),
  type: String (required),
  status: String (enum),
  room: ObjectId (ref: Room),
  devices: [ObjectId],
  ...
}
```

Document:

```

javascript
{
  patient: ObjectId (ref: PatientExtended),
  type: String (required),
  title: String,
  content: String,
  template: ObjectId (ref: DocumentTemplate),
  createdBy: ObjectId (ref: User),
  ...
}

```

Invoice:

```

javascript
{
  patient: ObjectId (ref: PatientExtended),
  doctor: ObjectId (ref: User),
  services: [{
    code: String,
    description: String,
    quantity: Number,
    unitPrice: Number,
    total: Number
  }],
  billingType: String (enum: ['kassenarzt', 'wahlarzt', 'privat']),
  totalAmount: Number,
  status: String (enum),
  ...
}

```

5.2 Indizes

Performance-Optimierung:

- Indizes auf häufig abgefragten Feldern
- Compound Indizes für komplexe Queries

- Text-Indizes für Volltextsuche

Beispiel:

```
javascript
PatientExtendedSchema.index({ lastName: 1, firstName: 1 });
AppointmentSchema.index({ startTime: 1, doctor: 1 });
```

6. API-Dokumentation

6.1 Authentifizierung

Login:

```
POST /api/auth/login
Body: { email: string, password: string }
Response: { token: string, user: User }
```

Registrierung:

```
POST /api/auth/register
Body: { email, password, firstName, lastName, role }
Response: { token: string, user: User }
```

Aktueller Benutzer:

```
GET /api/auth/me
Headers: { Authorization: "Bearer " }
Response: { user: User }
```

6.2 Patienten

Alle Patienten:

```
GET /api/patients?page=1&limit=20&search=...  
Response: { success: true, data: [Patient], pagination: {...} }
```

Patient erstellen:

```
POST /api/patients  
Body: Patient-Daten  
Response: { success: true, data: Patient }
```

Patient aktualisieren:

```
PUT /api/patients/:id  
Body: Patient-Daten  
Response: { success: true, data: Patient }
```

Patient löschen:

```
DELETE /api/patients/:id  
Response: { success: true }
```

6.3 Termine

Alle Termine:

```
GET /api/appointments?startDate=...&endDate=...  
Response: { success: true, data: [Appointment] }
```

Termin erstellen:

```
POST /api/appointments  
Body: Appointment-Daten  
Response: { success: true, data: Appointment }
```

Termin aktualisieren:

```
PUT /api/appointments/:id
Body: Appointment-Daten
Response: { success: true, data: Appointment }
```

6.4 Dokumente

Alle Dokumente:

```
GET /api/documents?patientId=...
Response: { success: true, data: [Document] }
```

Dokument erstellen:

```
POST /api/documents
Body: Document-Daten
Response: { success: true, data: Document }
```

Dokument als PDF:

```
GET /api/documents/:id/pdf
Response: PDF-Datei
```

6.5 Abrechnung

Rechnung erstellen:

```
POST /api/invoices
Body: Invoice-Daten
Response: { success: true, data: Invoice }
```

Rechnung an Versicherung senden:

```
POST /api/invoices/:id/submit
Response: { success: true, submissionId: string }
```

ÖGK XML Export:

```
GET /api/invoices/ogk-export?startDate=...&endDate=...  
Response: XML-Datei
```

6.6 Status-Codes

- `200 OK` : Erfolgreiche Anfrage
 - `201 Created` : Ressource erstellt
 - `400 Bad Request` : Ungültige Anfrage
 - `401 Unauthorized` : Nicht authentifiziert
 - `403 Forbidden` : Keine Berechtigung
 - `404 Not Found` : Ressource nicht gefunden
 - `500 Internal Server Error` : Server-Fehler
-

7. Sicherheit

7.1 Authentifizierung

JWT (JSON Web Tokens):

- Token-basierte Authentifizierung
- Token enthält Benutzer-ID und Rolle
- Token-Ablaufzeit: 24 Stunden (konfigurierbar)

- Refresh-Token für längere Sessions

Passwort-Sicherheit:

- Passwörter werden mit bcrypt gehasht
- Salt-Runden: 10
- Mindestlänge: 8 Zeichen
- Empfohlene Komplexität

7.2 Autorisierung

Rollenbasierte Zugriffskontrolle (RBAC):

- Rollen: Administrator, Arzt, Ordinationsgehilfe, Rezeption, Buchhaltung
- Berechtigungen pro Rolle
- Granulare Berechtigungen pro Ressource

ACL (Access Control List):

- Individuelle Berechtigungen pro Benutzer
- Ressourcen-spezifische Berechtigungen
- Vererbung von Rollen-Berechtigungen

7.3 Datenverschlüsselung

In Transit:

- HTTPS für alle Verbindungen
- TLS 1.2 oder höher

At Rest:

- Sensitive Daten verschlüsselt gespeichert
- Passwörter gehasht
- Versicherungsnummern verschlüsselt

7.4 Audit-Logging

Protokollierte Aktionen:

- Benutzeranmeldungen
- Datenänderungen (Create, Update, Delete)
- Dokumentenerstellung
- Rechnungserstellung
- Systemänderungen

Audit-Log Format:

```
javascript
{
  user: ObjectId,
  action: String,
  resource: String,
  resourceId: ObjectId,
  timestamp: Date,
  ipAddress: String,
  userAgent: String
}
```

8. Integrationen

8.1 ELGA

Elektronische Gesundheitsakte:

- ELGA-API Integration
- Dokumente in ELGA hochladen
- ELGA-Daten abrufen
- ELGA-Werte-Sets verwalten

Konfiguration:

- ELDA-Schnittstelle konfigurieren
- Test- und Produktiv-Modus
- Zertifikate verwalten

8.2 Versicherungen

ÖGK (Österreichische Gesundheitskasse):

- XML-Export im ELA-Format
- Automatische Übermittlung
- Tarifdatenbank-Download

Andere Kassen:

- SVS, BVAEB, KFA, PVA
- XML-Export-Funktion
- Versicherungsprüfung

Privatversicherungen:

- REST, FHIR, SOAP Integration
- E-Mail-Integration
- PDF-Export

8.3 DICOM

Digital Imaging:

- DICOM-Empfang
- DICOM-Versand

- DICOM-Viewer integriert
- OHIF Viewer Integration

8.4 Labor

Labor-Integration:

- Laboraufträge versenden
- Laborergebnisse empfangen
- Automatische Zuordnung zu Patienten
- HL7 FHIR Support

8.5 Zahlungen

Payment Gateway:

- Stripe Integration
- SEPA Lastschrift
- Kreditkartenzahlung
- Mobile Payment

9. Deployment

9.1 Produktionsumgebung

Server-Anforderungen:

- Node.js v16+
- MongoDB 4.4+
- Nginx (als Reverse Proxy)
- SSL-Zertifikat

Umgebungsvariablen:

```
env
NODE_ENV=production
PORT=5001
MONGODB_URI=mongodb://...
JWT_SECRET=strong-secret-key
```

9.2 Docker Deployment

Docker Compose:

```
yaml
version: '3.8'
services:
  backend:
    build: ./backend
    ports:
      - "5001:5001"

    environment:
      - MONGODB_URI=mongodb://mongo:27017/ordinationssoftware

  frontend:
    build: ./frontend
    ports:
      - "3000:80"

  mongo:
    image: mongo:4.4
    volumes:
      - mongo-data:/data/db
```

9.3 Build-Prozess

Frontend Build:

```
bash
cd frontend
npm run build
```

Erstellt optimierte Produktions-Bu

Backend:

```
bash
cd backend
npm start
```

Oder mit PM2 für Production:

```
pm2 start server.js
```

9.4 Nginx Konfiguration

```
nginx
server {
    listen 80;
    server_name your-domain.com;

    # Redirect to HTTPS
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl;
    server_name your-domain.com;

    ssl_certificate /path/to/cert.pem;
    ssl_certificate_key /path/to/key.pem;

    # Frontend
    location / {
        root /path/to/frontend/build;
        try_files $uri $uri/ /index.html;
    }

    # Backend API
    location /api {
        proxy_pass http://localhost:5001;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

10. Wartung und Updates

10.1 Backups

Automatische Backups:

- Täglich um 01:00 UTC
- Gespeichert in `backend/backups/`
- Format: `backup-YYYY-MM-DDTHH-MM-SS-XXXZ.tar.gz`

Manuelle Backups:

```
bash
cd backend
node scripts/backup.js
```

Backup wiederherstellen:

```
bash
cd backend
node scripts/restore.js backup-file.tar.gz
```

10.2 Updates

System-Updates:

- Automatische Updates für ICD-10 Katalog
- Automatische Updates für Tarifdatenbanken
- Manuelle System-Updates über Update-Monitoring

Update-Monitoring:

1. Gehen Sie zu "Update-Monitoring"
2. Prüfen Sie verfügbare Updates
3. Starten Sie Updates manuell oder automatisch

10.3 Logging

Log-Dateien:

- Backend: `backend.log`
- Frontend: Browser Console
- Nginx: `/var/log/nginx/`

Log-Level:

- ERROR: Fehler
- WARN: Warnungen
- INFO: Informationen
- DEBUG: Debug-Informationen

10.4 Monitoring

System-Monitoring:

- Server-Ressourcen (CPU, RAM, Disk)
- Datenbank-Performance
- API-Response-Zeiten
- Fehlerrate

Integration-Status:

- Status aller Integrationen

- Verbindungsprüfung
 - Fehlerbehandlung
-

Anhang

A. Glossar

API: Application Programming Interface

JWT: JSON Web Token

RBAC: Role-Based Access Control

ACL: Access Control List

ELGA: Elektronische Gesundheitsakte

ELDA: Elektronische Datenübertragung

DICOM: Digital Imaging and Communications in Medicine

FHIR: Fast Healthcare Interoperability Resources

HL7: Health Level Seven

REST: Representational State Transfer

SOAP: Simple Object Access Protocol

B. Nützliche Befehle

Backend:


```
bash
```

Server starten

```
npm start
```

Entwicklung mit Auto-Reload

```
npm run dev
```

Tests ausführen

```
npm test
```

Frontend:

```
bash
```

Development Server

```
npm start
```

Production Build

```
npm run build
```

Tests

```
npm test
```

Datenbank:

```
bash
```

MongoDB Shell

```
mongosh
```

Datenbank wechseln

```
use ordinationssoftware
```

Collections anzeigen

```
show collections
```

C. Support und Kontakt

Technischer Support:

- E-Mail: tech-support@mymedicloud.at
- Dokumentation: <https://docs.mymedicloud.at>
- GitHub Issues: [Repository URL]

Ende der Technischen Dokumentation

MyMediCloud MMC

Stand: Januar 2026

Version 1.0